MiniOS Alpha

Technical Documentation

Design, modules, testing and lessons learned

Group: Marquez and Friends

Operating Systems Final Project

Command-line OS simulator written in Python

Version 1.0

1. Introduction

MiniOS Alpha is a command-line operating system simulator built by Marquez and Friends for our Operating Systems final project. It is written in Python and runs inside Windows Command Prompt with the command `python main.py`.

MiniOS is not a real operating system. It does not boot, have a kernel, or control hardware. Instead it reproduces the behaviour of a small operating system on the screen: a user logs in, types commands, manages files, starts and stops programs, and watches memory being used. Everything happens inside Python objects, so the real computer is never affected.

This document explains how the system is organised, how each part works, how we tested it, and what we learned.

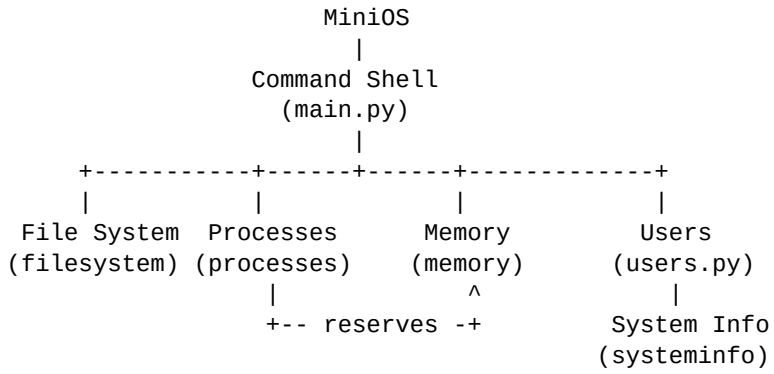
2. Project Objectives

Our goals for the project were:

- Build a working command-line OS simulator in Python that runs through Windows CMD.
- Provide all required features: a command prompt, help, file, process and memory management, user login, system information, error handling and exit.
- Cover the full minimum command list of 19 commands, and add a few useful extras (write, rename, cd, suspend, resume, users, logout, history, echo, version).
- Keep the code organised, with one Python file for each major OS concept.
- Show how the parts of an operating system depend on each other, for example how starting a process uses memory.
- Handle bad input politely so the program never crashes.
- Test every command and document how the system works.

3. System Architecture

MiniOS has one entry point, the command shell. The shell reads what the user types and passes each command to the part of the system that owns it. The diagram below shows the major components.



How the parts connect:

- The **shell** is the only part that talks to the user. It decides which module handles each command.
- **Processes** depend on **Memory**. Starting a process reserves 16 MB and killing it gives the 16 MB back.
- **Users** provides the login and the current user. **System Info** reads from all the other parts (user, memory and process count) to build one report.
- The **File System** is independent. It only needs its own data.

4. Program Structure

The project is split into five Python files. Each file has one job, which makes the code easier to explain, test and share between group members.

File	Main class	What it does
main.py	MiniOS	Starts the program, shows the banner, runs the login, reads commands, sends each one to the right handler, and prints help and systeminfo.
users.py	UserManager	Stores the accounts, checks passwords, runs the login screen, tracks the current user and role.
filesystem.py	FileSystem, Node	Holds the virtual folders and files and the current directory. Handles dir, mkdir, create, read, write, delete, rename and cd.
processes.py	ProcessManager, Process	Keeps the process table. Handles run, kill, suspend, resume and process.
memory.py	MemoryManager	Tracks the 1024 MB of simulated memory. Handles allocate, free and memory.

Other files in the project:

- **README.md**: how to run MiniOS, the accounts and the command list.
- **requirements.txt**: confirms that only the Python standard library is needed.
- **docs/**, **screenshots/**, **individual-reports/**: documentation, evidence and reports.

How a command is processed

Every command goes through the same steps in main.py:

- The text is split into words with `shlex.split`, so names in quotes stay together.
- The first word is changed to lowercase and looked up in a dictionary of commands.
- If the command is found, its method runs. If not, MiniOS prints 'Unknown command.'
- The method calls the right module, checks the arguments, and prints the result.

We used a dictionary instead of a long chain of if and elif statements. To add a new command, a member writes one method and adds one line to the dictionary.

5. File Management

The file system is virtual. Nothing is written to the real disk, so the real computer is safe and the simulation is easy to explain. Everything lives in memory and is gone when MiniOS exits.

How it works

The file system is a tree of Node objects. Each Node stores its name, whether it is a directory or a file, a link to its parent, its text content, and (for directories) a dictionary of the items inside it. The FileSystem class keeps a root node and a pointer to the current directory.

- **dir** lists the items in the current directory, folders first, with file sizes in bytes.
- **mkdir** and **create** add a new directory or empty file under the current directory.
- **read** and **write** show or replace a file's text.
- **delete** removes a file or an empty directory.
- **rename** changes a name while keeping the contents.
- **cd** moves the current directory pointer. It understands relative paths, absolute paths and `..` for the parent folder.
- The prompt shows the current path by following parent links back to the root.

Rules the file system enforces

- Names cannot be empty, cannot be . or .., and cannot contain the characters \ / : * ? " < > |
- Two items in the same directory cannot share a name.
- read and write refuse directories. cd refuses files.
- A directory that still contains items cannot be deleted.

6. Process Management

Processes are simulated. MiniOS does not start any real Windows programs. A process is only a record in a table, but it behaves like a simple process control block.

How it works

- Each Process stores a PID, a name, a status, the memory it uses and its start time.
- The ProcessManager keeps all processes in a dictionary that uses the PID as the key.
- PIDs start at 1 and only go up, so a PID is never reused during a session.
- **run** accepts only a fixed list of programs: calculator, editor, notepad, browser, terminal and clock.
- **process** prints the table with PID, name, status and memory.
- **kill** removes the process from the table.
- **suspend** and **resume** switch the status between RUNNING and SUSPENDED.
- On exit or logout, every process is killed so that nothing is left behind.

Link to memory

Each process uses 16 MB of simulated memory. When `run` is called, the ProcessManager asks the MemoryManager for 16 MB. If there is not enough free memory, the process is not created and an error is shown. When the process is killed, the 16 MB is returned. This shows how an operating system accounts for the resources that programs use.

7. Memory Management

Memory is simulated with simple counters. MiniOS never touches the real RAM. The simulated computer has 1024 MB in total.

How it works

Used memory is the sum of three parts:

Part	Starts at	Changes when
System memory	256 MB	Never. It stands for the kernel and background services.
User-allocated memory	0 MB	The user runs allocate or free.
Process memory	0 MB	A process is started or killed (16 MB each).

- Free memory is 1024 minus used memory.
- **memory** prints total, used and free memory.
- **allocate** refuses amounts that are zero, negative or bigger than the free memory.
- **free** refuses amounts bigger than what the user has allocated. This stops a user from releasing memory that belongs to the system or to running processes.

Example: at start-up used memory is 256 MB. After `allocate 200` it is 456 MB. After `free 200` it is back to 256 MB. Each running process adds another 16 MB on top of this.

8. User Management

MiniOS has a login system, so every session starts by proving who is using the system.

How it works

- Two accounts are built in: **admin** (Administrator) and **student** (Standard User).
- Passwords are not stored as plain text. The UserManager stores a SHA-256 hash of each password and compares hashes at login.
- The password prompt uses getpass so typing is hidden. If the terminal does not support that, it falls back to normal input.
- The user gets three attempts. After the third failure the login screen starts again.
- After a successful login MiniOS prints 'Login successful.' and 'Welcome, username!'
- **whoami** shows the current user. **users** lists the accounts. **logout** ends the session, stops all processes and returns to the login screen.

- **systeminfo** reports the OS name, version, architecture, virtual CPU, memory, the user with their role, and the number of running processes.

Limitation: an unsalted SHA-256 hash is acceptable for a classroom simulation. A real system would use a salted, slow hashing method.

9. Error Handling

MiniOS is designed so that bad input never crashes the program. Errors are handled at three levels.

Level	What it catches	Example response
Shell	Unknown commands, missing arguments, text where a number is needed, unbalanced quotes	Unknown command. Type 'help' to see the available commands.
Modules	Rule violations such as missing files, duplicate names, unknown PIDs, not enough memory	Error: File does not exist. / Error: Process does not exist.
Safety net	Any unexpected exception during a command	Error: Something went wrong (details).

More examples of the messages a user can see:

- `hellooo` gives 'Unknown command.' followed by a hint to type help.
- `read missing.txt` gives 'Error: File does not exist.'
- `kill 999` gives 'Error: Process does not exist.'
- `kill abc` gives 'Error: PID must be a whole number.'
- `allocate 5000` gives a message with how much memory is actually free.
- A command with missing arguments prints a Usage line, for example 'Usage: mkdir <name>'.
- Ctrl+C or end-of-input leads to a normal shutdown instead of an error screen.

10. Testing

We tested MiniOS by running it from the command line and typing commands, and by feeding it a prepared list of commands, one full command per line, so the same session could be repeated after each fix. We tested three kinds of input for each feature:

- **Normal input:** the command used the correct way.
- **Error input:** missing files, unknown PIDs, wrong passwords, oversized memory requests.
- **Edge cases:** zero and negative numbers, duplicate names, names with special characters, unbalanced quotes, and moving above the root directory.

We also compared the screens with the examples in the project sheet (login screen, memory status, systeminfo and process list) to make sure the formats match. The table shows the main tests and what we observed.

Area	Input	Observed result	Result
Login	wrong password	Invalid username or password, 2 attempts left	Pass
Login	three wrong passwords	Login screen starts again	Pass

Area	Input	Observed result	Result
Login	admin / admin123	Login successful, Welcome, admin!	Pass
Errors	hellooo	Unknown command message	Pass
Errors	echo "unbalanced	Error: Unbalanced quotes in command.	Pass
Files	mkdir Documents	Directory created.	Pass
Files	create, write, read notes.txt	File created, File updated, content shown	Pass
Files	read missing.txt	Error: File does not exist.	Pass
Files	delete Documents (has a file)	Error: Directory is not empty.	Pass
Files	duplicate mkdir, name with *	Already exists error, Invalid directory name error	Pass
Files	rename, then delete the file	Renamed. File deleted.	Pass
Process	run calculator, run editor	PID 1 and PID 2 started	Pass
Process	run foo	Error: Program not found.	Pass
Process	suspend 1, resume 1	Status changes to SUSPENDED then RUNNING	Pass
Process	kill 999 and kill abc	Process does not exist. PID must be a whole number.	Pass
Memory	memory with one process running	Used 272 MB (256 system + 16 process)	Pass
Memory	allocate 200, then memory	200 MB allocated, used rises by 200	Pass
Memory	allocate 5000	Error: Not enough free memory.	Pass
Memory	allocate -5, allocate 0, free -3	Error: Amount must be greater than zero.	Pass
Memory	free 200, then free 999	200 MB released. Then error about the allocated amount	Pass
Users	users, whoami, logout, login as student	Correct output, second login works	Pass
System	systeminfo, exit	Info shown. Shutting down MiniOS... Goodbye!	Pass

11. Problems Encountered

These are the main problems that came up while building and testing MiniOS, and how we solved them.

Problem	How we solved it
Commands with names that contain spaces were split into separate words.	We parse input with shlex.split so quoted names stay together, and print a clear error if the quotes are not closed.
The password prompt showed a warning and echoed the typing when the terminal was not a normal console.	We use getpass first and fall back to plain input if it fails. Normal Command Prompt use hides the password.

Problem	How we solved it
Killing a process had to give its memory back, but a user must not be able to free memory that belongs to a process.	We split memory into three counters (system, user and process). allocate and free only touch the user counter, and the ProcessManager uses its own reserve and release methods.
Processes could be left running after logout or exit, keeping memory reserved.	We added a kill_all method that runs on exit and on logout.
The clear command works differently on different systems.	main.py checks the operating system and runs cls on Windows and clear elsewhere.
Deleting a folder could silently throw away files inside it, and cd could walk above the root.	delete refuses non-empty directories, and the path code stops at the root.
Our first test run gave misleading results because the prepared command list had each word on its own line, so commands never received their arguments.	We changed the test file to hold one full command per line and re-ran every test. The shell itself was working correctly.
Any unexpected bug could end the whole program.	Every command runs inside a try/except in the shell, so an unexpected error prints a message and the prompt returns.

12. Conclusion

Building MiniOS helped Marquez and Friends understand how an operating system is put together. Writing each part ourselves showed that an OS is a set of cooperating services behind a shell, not one single program.

- **The shell** is the link between a person and the system. Good parsing and clear messages make it usable.
- **File systems** are trees, and many small rules (unique names, non-empty directories, path resolution) are needed to keep them consistent.
- **Processes** need identity (a PID), a state, and resources. The link between processes and memory showed why an OS must account for what programs use.
- **Memory management** is about limits. Every request must be checked against what is free.
- **User management** starts with authentication, and passwords should be stored as hashes.
- **Error handling and testing** are part of the design. Checking bad input and edge cases found problems that normal use would never show.

We also learned how to divide a project by module, agree on how the modules talk to each other, and keep the code simple enough that every member can explain it. Possible future work includes saving the file system to disk, using the roles to restrict commands, adding a simple CPU scheduler, and storing passwords with a salted hash.